T.C.
Pamukkale Üniversitesi
İktisadi ve İdari Bilimler Fakültesi
Yönetim Bilişim Sistemleri Bölümü

STAJ RAPORU

Öğrencinin numarası	:	[Öğrenci Numaranız]
Adı soyadı	:	[Adınız Soyadınız]
Yapılan staj türü	:	Yazılım Stajı (Zorunlu)
Yapılan staj iş günü sayısı	:	**30**
Staj tarihi	:	**16/09/2026 - 27/10/2026**

T.C.
Pamukkale Üniversitesi
İktisadi ve İdari Bilimler Fakültesi
Yönetim Bilişim Sistemleri Bölümü

Öğrencinin numarası	:	[Öğrenci Numaranız]
Adı soyadı	:	[Adınız Soyadınız]
Yapılan staj türü	:	Yazılım Stajı (Zorunlu)
Yapılan staj iş günü sayısı	:	**30**
Staj tarihi	:	**16/09/2026 - 27/10/2026**

.

T.C.
Pamukkale Üniversitesi
İktisadi ve İdari Bilimler Fakültesi
Yönetim Bilişim Sistemleri Bölümü

Öğrencinin Adı Soyadı	:	[Adınız Soyadınız]
Numarası	:	[Öğrenci Numaranız]
Mobil Telefon No	:	[Mobil Telefon Numaranız]
Yapılan Staj Türü	:	Yazılım Stajı (Zorunlu)
Staj Tarihi	:	**16/09/2026 - 27/10/2026**

Fotoğraf

İşyeri staj sorumlusunun dikkatine!

30 iş günü olarak yapılacak olan bu stajda öğrenciler, stajları boyunca her gün aktivitelerini staj defterlerine işleyeceklerdir. Staj bitiminde işverenleri tarafından hazırlanan staj değerlendirmelerini staj defterleri ve raporları ile bölüme ileteceklerdir.

Staj yapılan işyeri adı	:	[Firma / Kurum Adı]
Adresi	:	[Firma Adresi]
İşyeri staj sorumlusunun Ünvanı, adı, soyadı	:	[Yetkili Adı Soyadı] — Yazılım Takım Lideri
Tarih	:	27/10/2026
İmza	:	
Kaşe	:	

Haftalık Staj Özeti

Haftanın Tarihleri 16/09/2026 - 18/09/2026

Gün	Yapılan İş	Sayfa
Pazartesi	-	-
Salı	-	-
Çarşamba	Staj Oryantasyonu, Bilgisayar Kurulumları ve Proje Planlaması	1
Perşembe	Veritabanı Tasarımı ve Tablo İlişkilerinin Çıkarılması (ERD)	2
Cuma	ASP.NET Core Web API İskeleti ve Entity Modelleri	3
Cumartesi	- (Hafta Sonu)	-

Haftalık Staj Özeti

Haftanın Tarihleri 21/09/2026 - 25/09/2026

Gün	Yapılan İş	Sayfa
Pazartesi	Entity Framework Core Migration ve Generic Repository Yapısı	4
Salı	Gemi ve Liman Servisleri ile IMO Doğrulama Algoritması	5
Çarşamba	Sefer Ziyaretleri ve Yük Servislerindeki Mantık Kontrolleri	6
Perşembe	Mürettebat Servisi, Controller Katmanı ve Swagger API Testleri	7
Cuma	React + Vite Frontend Kurulumu ve Temel Arayüz Bileşenleri	8
Cumartesi	- (Hafta Sonu)	-

Haftalık Staj Özeti

Haftanın Tarihleri 28/09/2026 - 02/10/2026

Gün	Yapılan İş	Sayfa
Pazartesi	Frontend Sayfalarının Birleştirilmesi ve 1. Aşama Yönetici Demosu	**9**
Salı	2. Aşama: Büyük Hacimli Test Verisi Üretici (Seeder) Yazılması	**10**
Çarşamba	Entity Framework Core N+1 Sorgu Probleminin Çözümü	**11**
Perşembe	Çok Adımlı Operasyonlarda Atomik Transaction Yönetimi (Rollback)	**12**
Cuma	Eş Zamanlı Güncelleme Çakışmaları (RowVersion) ve Soft Delete	**13**
Cumartesi	- (Hafta Sonu)	**-**

Haftalık Staj Özeti

Haftanın Tarihleri 05/10/2026 - 09/10/2026

Gün	Yapılan İş	Sayfa
Pazartesi	3. Aşama: Web Güvenliği, OWASP Top 10 Analizi (RBAC & IDOR)	**14**
Salı	SQL Injection ve XSS Açıklarının İncelenmesi ve Çözülmesi	**15**
Çarşamba	Hassas Bilgi Sızıntısı ve Mass Assignment Önleme	**16**
Perşembe	Güvenli Kimlik Doğrulama (JWT & Refresh Token) ve Rate Limiting	**17**
Cuma	Güvenli Dosya Yükleme, Audit Logging ve Güvenlik Laboratuvarı	**18**
Cumartesi	- (Hafta Sonu)	**-**

Haftalık Staj Özeti

Haftanın Tarihleri 12/10/2026 - 16/10/2026

Gün	Yapılan İş	Sayfa
Pazartesi	Akıllı Sefer Planlama Sihirbazı ve Safe Manning Kuralları	**19**
Salı	Canlı Deniz Haritası (Live Map) ve Gerçek Rota Koordinatları	**20**
Çarşamba	Frontend Performans İyileştirmeleri, Kod Bölme ve Hook Optimizasyonu	**21**
Perşembe	Kargo Yönetimi Mantık Düzenlemeleri ve Liman Kontrolleri	**22**
Cuma	Raporlama Modülü, İstatistik Göstergeleri ve CSV Dışa Aktarımı	**23**
Cumartesi	- (Hafta Sonu)	**-**

Haftalık Staj Özeti

Haftanın Tarihleri 19/10/2026 - 27/10/2026

Gün	Yapılan İş	Sayfa
Pazartesi	Kurumsal Güvenlik Sıkılaştırması: API Okuma Uç Noktaları	**24**
Salı	JWT Anahtarının Güvenli Yapılandırılması ve Ortam İzolasyonu	**25**
Çarşamba	Yetkilendirme ve Güvenlik Birim Testlerinin Yazılması (144 Test)	**26**
Perşembe	Kullanıcı Deneyimi İyileştirmeleri ve İskelet Yükleyiciler	**27**
Cuma	Tarayıcı Testleri, Kod Temizliği ve Çalışma Alanı Eşitlemesi	**28**
Cumartesi	26-27 Ekim: Dokümantasyon, Final Staj Savunması ve Teslim	**29-30**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Staj Oryantasyonu, Bilgisayar Kurulumları ve Proje Planlaması** Tarih: **16.09.2026**

Bugün stajımın ilk günü olması sebebiyle kurum içi oryantasyon ve geliştirme ortamının hazırlanması süreçleriyle güne başladım. Sabah saatlerinde kurumdaki staj amirim ve yazılım ekibindeki kıdemli mühendislerle tanıştım. Ekip lideri bana şirketin çalışma kültürü, kullanılan kurumsal çevik (Agile/Scrum) yöntemleri, kod inceleme (Code Review) süreçleri ve Git versiyon kontrol sistemi politikaları hakkında kapsamlı bir oryantasyon sağladı.

Staj süresince geliştireceğim 'Liman Gemi Takip Sistemi' projesinin ihtiyaç analiz dokümanını (STAJ_PROJE_PLANI.md) staj sorumlum ile birlikte detaylıca inceledik. Projenin temel hedefi; limana gelen ve ayrılan gemilerin, yük manifestolarının, sefer ziyaret kayıtlarının, rıhtım kapasitelerinin ve gemi personelinin uluslararası denizcilik kurallarına uygun olarak tek bir merkezi platformdan yönetilmesidir. Yapılan toplantıda projenin 5 ana fonksiyonel modülden (Gemi Yönetimi, Liman Yönetimi, Sefer Takibi, Kargo/Yük Yönetimi ve Mürettebat Atamaları) oluşmasına karar verdik.

Öğleden sonraki süreçte geliştirme yapacağım bilgisayarın çalışma ortamını sıfırdan kurdum. Geliştirme ortamına .NET 8.0 SDK, Visual Studio Code, Node.js (v20+), npm paket yöneticisi, ilişkisel veritabanı yönetimi için SQL Server Express ve sorgulama arayüzü olarak Azure Data Studio kurulumlarını tamamladım. Ayrıca şirket reposuna erişebilmek için Git kimlik bilgilerimi ve SSH anahtarlarımı yapılandırdım.

Kurulumların ardından terminalden örnek bir .NET Web API ve Node ortamı ayağa kaldırarak tüm geliştirme araçlarının, derleyicilerin ve paket yöneticilerinin sorunsuz çalıştığını doğruladım. Yarın başlayacağım ilişkisel veritabanı ve ERD tasarımı için ön hazırlıklarımı tamamlayarak günü sonlandırdım.

Günün Çıktısı:
Şirket içi oryantasyon tamamlandı, geliştirme ortamı araçları eksiksiz kuruldu ve Liman Gemi Takip Sistemi'nin iş gereksinimleri analiz edildi.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Veritabanı Tasarımı ve Tablo İlişkilerinin Çıkarılması (ERD)** Tarih: **17.09.2026**

Bugün projenin veritabanı şemasını ve varlık-ilişki diyagramını (ERD) tasarlamaya başladım. Liman lojistiği ve denizcilik operasyonlarında nelerin takip edilmesi gerektiğini kağıt üzerinde ve SQL Server diyagram aracında modelledim. Sistem için 6 ana varlık belirledim: Ships (Gemiler), Ports (Limanlar), ShipVisits (Sefer Ziyaretleri), Cargoes (Kargolar), CrewMembers (Personel) ve ShipCrewAssignments (Görevlendirmeler).

Tablolar arasındaki Primary Key (PK) ve Foreign Key (FK) bağlantılarını kurdum. Bir geminin zaman içinde birden fazla limanı ziyaret edebileceği ve üzerinde farklı yükler taşıyabileceği için Ships tablosundan ShipVisits ve Cargoes tablolarına bire-çok (1:N) ilişkiler tanımladım. Benzer şekilde bir limanın aynı anda birden fazla gemiyi ağırlayabilmesi için Ports ile ShipVisits arasında 1:N bağlantı oluşturdum.

Personel yönetiminde ise bir denizcinin farklı tarihlerde farklı gemilerde çalışabilmesi gerektiğinden, Ships ile CrewMembers arasındaki çoka-çok (N:M) ilişkiyi ShipCrewAssignments ara tablosu ile normalize ettim. Veri bütünlüğünü korumak adına zorunlu alanları, IMO numarası benzersizlik kuralını, pozitif ağırlık kısıtlamalarını ve UN/LOCODE standart liman kodlarını belirledim.

Günün sonunda tasarladığım şemayı Mermaid sözdizimiyle çizerek 'docs/DATABASE_DIAGRAM.md' adıyla proje dokümantasyonuna ekledim ve staj amirimin onayına sundum.

Günün Çıktısı:
Veritabanı 3NF normalizasyon kurallarına göre tasarlandı, tablo ilişkileri çıkarıldı ve ERD dokümanı hazırlandı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **ASP.NET Core Web API İskeleti ve Entity Modellerinin Kodlanması** Tarih: **18.09.2026**

Bugün projenin backend tarafını geliştirmeye resmi olarak başladım. Visual Studio terminalinden .NET CLI komutlarını kullanarak .NET 8.0 tabanlı 'LimanGemiTakip.Api' isimli Web API projesini açtım.

Projenin temiz, sürdürülebilir ve genişletilebilir olması amacıyla N-Katmanlı Mimari (N-Tier Architecture) klasör yapısını kurguladım. Çözüm içerisine Models, Data, Repositories, Services, Controllers ve DTOs klasörlerini oluşturdum. Dün tasarladığım veritabanı tablolarını C# sınıfları (Entity) olarak kodladım: Ship.cs, Port.cs, ShipVisit.cs, Cargo.cs, CrewMember.cs ve ShipCrewAssignment.cs sınıflarını yazdım.

Modellerin içerisine navigation property'leri, Foreign Key özniteliklerini ve Data Annotations kurallarını ([Required], [MaxLength]) yerleştirdim. Entity Framework Core bağımlılıklarını (Microsoft.EntityFrameworkCore.SqlServer ve Tools) NuGet paket yöneticisiyle projeye dahil ettim.

Ardından ApplicationDbContext sınıfını yazarak veritabanı bağlamını oluşturdum ve tüm modellerimi DbSet<T> özellikleri olarak tanımladım. Program.cs dosyasına DbContext servisini ekleyerek backend altyapısının temelini başarıyla attım.

Günün Çıktısı:
ASP.NET Core 8 Web API projesi açıldı, katmanlı mimari klasörleri düzenlendi ve tüm Entity modelleri DbContext ile bağlandı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Entity Framework Core Migration ve Generic Repository Yapısı** Tarih: **21.09.2026**

Yeni haftanın ilk gününde veritabanı bağlantısı ve veri erişim katmanı (Data Access Layer) üzerinde çalıştım. İlk olarak appsettings.json dosyasına SQL Server Express yerel sunucusuna bağlanacak DefaultConnection bağlantı dizesini ekledim.

Entity Framework Core Code-First yaklaşımıyla terminalden 'dotnet ef migrations add InitialCreate' komutunu çalıştırarak ilk migration dosyamı ürettim. Oluşan C# migration dosyasındaki Up ve Down metodlarını inceledikten sonra 'dotnet ef database update' komutuyla tabloların SQL Server'da fiziksel olarak oluşturulmasını sağladım. Azure Data Studio üzerinden veritabanına bağlanarak tüm tabloların, indekslerin ve kısıtların eksiksiz açıldığını teyit ettim.

Veri erişim işlemlerinde kod tekrarını önlemek ve servis katmanını veritabanı teknolojisinden soyutlamak için Generic Repository tasarım desenini uyguladım. IRepository<T> generic arayüzünü ve bu arayüzü uygulayan Repository<T> sınıfını kodladım. GetAllAsync, GetByIdAsync, FindAsync, AddAsync, UpdateAsync ve DeleteAsync temel CRUD metodlarını yazdım.

Program.cs içerisinde AddScoped ile IRepository arayüzünün bağımlılık enjeksiyonunu (DI) yapılandırarak veri erişim katmanını tamamladım.

Günün Çıktısı:
Code-First migration ile veritabanı fiziksel olarak üretildi ve Generic Repository deseni başarıyla kodlandı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Gemi ve Liman Servisleri ile IMO Doğrulama Algoritması** Tarih: **22.09.2026**

Bugün projenin iş mantığı katmanını (Business Logic Layer) geliştirmeye geçtim. Gemi yönetimi için IShipService ve ShipService sınıflarını, liman yönetimi için IPortService ve PortService sınıflarını kodladım.

Gemi kaydı oluştururken en kritik sektörel kural olan 7 basamaklı IMO (Uluslararası Denizcilik Örgütü) numarasının doğruluğunu sağlayan algoritmayı araştırdım ve C# ile kodladım. Algoritmaya göre: ilk 6 rakam sırasıyla 7, 6, 5, 4, 3 ve 2 katsayılarıyla çarpılıp toplanmakta ve toplamın 10'a bölümünden kalan (mod 10) değerinin son basamağa (7. rakama) eşit olması gerekmektedir. Bu checksum kontrolünü servis katmanına entegre ettim; kurala uymayan veya mükerrer IMO girildiğinde sistemin açıklayıcı bir ArgumentException fırlatmasını sağladım.

Liman servisinde ise liman adının boş geçilememesi ve Birleşmiş Milletler UN/LOCODE formatına uygun 5 karakterli liman kodunun (örneğin TRIST, TRMER, TRKOP) veritabanında benzersiz olması kuralını yazdım. Var olan bir kod girildiğinde ekleme işlemini engelleyen validasyonları tamamladım.

Günün Çıktısı:
Gemi ve Liman servisleri yazıldı, 7 haneli IMO Checksum algoritması ve UN/LOCODE benzersizlik kontrolleri başarıyla uygulandı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Sefer Ziyaretleri ve Yük Servislerindeki Mantık Kontrolleri** Tarih: **23.09.2026**

Bugün gemi ziyaretleri (seferler) ve kargo operasyonlarının servis katmanını kodladım: IShipVisitService/ShipVisitService ve ICargoService/CargoService sınıflarını geliştirdim.

Liman operasyonlarında sahada karşılaşılabilecek mantıksal hataları engellemek adına katı iş kuralları yazdım:
1. Geliş tarihi (ArrivalDate) ayrılış tarihinden (DepartureDate) kesinlikle önce olmak zorundadır.
2. Aynı geminin aynı tarih aralığında iki farklı limanda bulunması (Double Booking) engellenmiştir. Sistem mevcut aktif seferleri kontrol ederek çakışma durumunda işlemi reddetmektedir.
3. İlgili limanın fiziki yanaşma kapasitesi dolduğunda (PortCapacityFull) yeni gemi yanaşmasına izin verilmemektedir.

Kargo servisinde ise yük ağırlığının sıfırdan büyük olma zorunluluğunu ve bir gemiye yüklenen kargoların toplam ağırlığının geminin azami taşıma kapasitesini (Deadweight Tonnage - DWT) aşamayacağı kuralını doğruladım. DWT kapasitesini aşan yükleme taleplerinde sistemin işlemi durdurmasını sağladım.

Günün Çıktısı:
Sefer çakışması önleme, liman kapasite kontrolü ve gemi DWT yük sınırı doğrulama mekanizmaları servis katmanında tamamlandı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Mürettebat Servisi, Controller Katmanı ve Swagger API Testleri** Tarih: **24.09.2026**

Bugün mürettebat ve görev atamaları için ICrewMemberService ve IShipCrewAssignmentService sınıflarını yazdım. Bir denizcinin aktif bir görevi varken aynı tarih aralığında başka bir gemiye atanamaması ve görev bitiş tarihinin başlangıçtan sonra olması kurallarını kodladım.

Geliştirdiğim tüm iş servislerini HTTP üzerinden dış dünyaya açmak için Controllers klasöründe RESTful API uç noktalarını kodladım: ShipsController, PortsController, ShipVisitsController, CargoesController ve CrewMembersController sınıflarını oluşturdum. Tüm controller'larda ortak olan GetAll, GetById, Create, Update ve Delete operasyonları için generic CrudControllerBase taban sınıfını yazarak kod tekrarını önledim.

Program.cs dosyasına Swagger OpenAPI entegrasyonunu tamamladım. Tarayıcı üzerinden 'http://localhost:5000/swagger' adresine giderek oluşturduğum tüm API uç noktalarını tek tek test ettim. Gemi ekleme, sefer oluşturma ve hata durumlarında dönen HTTP durum kodlarını (200 OK, 201 Created, 400 Bad Request, 404 Not Found) doğruladım.

Günün Çıktısı:
Mürettebat servisleri ve tüm RESTful API controller uç noktaları yazıldı, Swagger UI üzerinden testleri başarıyla gerçekleştirildi.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **React + Vite Frontend Projesinin Kurulumu ve Temel Arayüz Bileşenleri** Tarih: **25.09.2026**

Backend API'si tamamlanan projenin bugün kullanıcı arayüzü (frontend) geliştirme adımına başladım. Terminalden 'npm create vite@latest frontend -- --template react-ts' komutuyla modern React 18 ve TypeScript tabanlı SPA projesi açtım.

Hızlı ve kurumsal bir tasarım geliştirebilmek için TailwindCSS kütüphanesini projeye dahil ettim. Denizcilik sektörünün ciddiyetine uygun lacivert, mavi ve nötr gri tonlarından oluşan bir tema paleti yapılandırdım. İkon desteği için Lucide-React kütüphanesini kurdum.

Backend ile HTTP haberleşmesini yönetmek üzere Axios kütüphanesini ekledim. src/services/api.ts dosyasını oluşturarak merkezi baseURL yapılandırmasını ve gelen hataları yakalayan response interceptor mekanizmasını kurdum.

Tüm sayfalarda kullanılacak ortak arayüz bileşenlerini modüler olarak kodladım: Üst navigasyon barı ve sol menüyü içeren Layout.tsx, listeleme tabloları için Table.tsx, açılır pencereler için Modal.tsx, durum bildirimleri için Badge.tsx ve buton bileşenlerini geliştirdim.

Günün Çıktısı:
React 18 + Vite + TypeScript frontend projesi kuruldu, TailwindCSS teması oluşturuldu ve merkezi Layout bileşenleri kodlandı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Frontend Sayfalarının Birleştirilmesi ve 1. Aşama Yönetici Demosu** Tarih: **28.09.2026**

Bugün frontend sayfalarını tamamlayarak backend API servisleriyle uçtan uca bağladım. src/pages/ altında ShipsPage.tsx, PortsPage.tsx, ShipVisitsPage.tsx, CargoPage.tsx ve CrewPage.tsx ekranlarını geliştirdim.

Her sayfaya dinamik arama çubuğu, kayıt listeleme tablosu, yeni kayıt ekleme modal formu ve silme/düzenleme işlevlerini bağladım. Formlarda eksik veya hatalı veri girildiğinde kullanıcıyı uyaran form validasyonlarını tamamladım. Backend test projesindeki 75 birim testinin tamamının yeşil yandığını terminalden doğruladım.

Öğleden sonra staj amirime ve şirket yöneticisine projenin 1. aşama canlı sunumunu yaptım. 'docs/YONETICI_SUNUM_REHBERI.md' dokümanında belirttiğim demo senaryosu üzerinden; gemi tanımlama, liman kapasitesi kontrolü, çakışmasız sefer planlama ve yükleme adımlarını canlı olarak gösterdim.

Staj amirim mimarinin temizliğini ve projenin iş kurallarına uyumunu takdir ederek 1. aşamayı onayladı ve 2. aşamaya geçiş izni verdi.

Günün Çıktısı:
Frontend CRUD ekranları tamamlandı, 75 birim testi doğrulandı ve 1. Aşama yönetici sunumu başarıyla gerçekleştirildi.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **2. Aşama: Büyük Hacimli Test Verisi Üretici (Data Seeder) Yazılması** Tarih: **29.09.2026**

Bugün staj çalışmasının 2. aşamasına resmi olarak başladık. Bu aşamanın odağı: Entity Framework Core performans optimizasyonu, büyük veri hacimlerinde sorgu analizi ve veri tutarlılığı idi. Staj amirim, sistemin gerçek hayatta olduğu gibi binlerce kayıt altındayken nasıl tepki vereceğini görmemizi istedi.

Bu doğrultuda C# ile DataSeederService adında otomatik ilişkisel test verisi üreten bir servis yazdım. Üretilen verilerin rastgele metinler yerine gerçekçi denizcilik verilerinden oluşmasını sağladım: Geçerli 7 basamaklı IMO algoritmasına sahip 1.050 gemi, gerçek Akdeniz/Karadeniz UN/LOCODE kodlu 120 liman, tarihleri mantıklı 5.100 sefer, 10.200 kargo ve 15.200 mürettebat ataması kodladım.

Toplamda yaklaşık 36.800 adet ilişkisel kaydı toplu ekleme (Bulk Insert) teknikleri kullanarak SQL Server veritabanına başarıyla yükledim. Sistemin veri tabanı boyutunu ve indeks doluluklarını Azure Data Studio üzerinden analiz ettim.

Günün Çıktısı:
Gerçekçi veri üreten DataSeederService geliştirildi ve veritabanına 36.800'den fazla ilişkisel test verisi yüklendi.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Entity Framework Core N+1 Sorgu Probleminin Çözümü** Tarih: **30.09.2026**

Bugün yüklediğimiz 36 bin kayıt ile gemi ve sefer listeleme sayfalarını açtığımda arayüzde belirgin bir yavaşlık fark ettim. Sorunun kaynağını anlamak için SQL Server Profiler ve ASP.NET Core EF Core detaylı konsol loglarını açarak arka planda veritabanına gönderilen SQL sorgularını inceledim.

İnceleme sonucunda; 50 gemiyi listelemek isterken her bir geminin seferlerini döngü içerisinde çektiğim için önce 1 gemi sorgusu, ardından her gemi için ayrı ayrı 50 sefer sorgusu olmak üzere toplam 1 + 50 = 51 adet SQL sorgusu atıldığını tespit ettim. Bunun literatürde 'N+1 Query Problemi' olarak bilinen klasik bir ORM performans tuzağı olduğunu öğrendim.

Bu sorunu çözmek için servis katmanındaki sorguları Eager Loading (.Include(s => s.ShipVisits)) ile tek bir SQL JOIN sorgusuna dönüştürdüm. Ayrıca sadece okuma yapılan listeleme sorgularına .AsNoTracking() ekleyerek EF Core'un Change Tracker yükünü kaldırdım.

Optimizasyon sonucunda SQL sorgu sayısı 51'den 1'e düştü, ağ üzerinden taşınan veri boyutu %98 azaldı ve sayfa yanıt süresi 15 kat hızlandı.

Günün Çıktısı:
N+1 sorgu problemi tespit edilip Eager Loading ve AsNoTracking ile çözüldü, veritabanı performansı 15 kat artırıldı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Çok Adımlı Operasyonlarda Atomik Transaction Yönetimi (Rollback)** Tarih: **01.10.2026**

Bugün veritabanı veri tutarlılığı ve ACID prensipleri kapsamında Atomik Transaction yönetimini projeye entegre ettim. Liman operasyonlarında bir gemi limana yanaştığında arka arkaya 3 kritik işlem yapılmaktadır: Sefer durumu 'Docked' olarak güncellenir, gemideki yükler sisteme işlenir ve gemi personeline liman iniş izni atanır.

Eğer bu çok adımlı sürecin 3. adımında bir veritabanı veya sistem hatası oluşursa, ilk iki adımın veritabanında kalması ciddi bir veri kirliliği ve operasyonel tutarsızlık yaratmaktadır. Bu durumu çözmek için EF Core IDbContextTransaction arayüzünü kullanarak servis katmanında atomik işlem yönetimi geliştirdim.

BeginTransactionAsync() ile transaction başlatıp, tüm adımlar başarılı olduğunda CommitAsync(), herhangi bir hata durumunda ise RollbackAsync() çalıştıran güvenli try-catch bloğu kodladım.

Yazdığım transaction yapısını test etmek için TransactionTests.cs sınıfında bir birim testi yazdım. İkinci adımda kasıtlı bir istisna (simulated error) fırlatarak sistemin ilk adımda eklenen sefer kaydını da veritabanından güvenle geri aldığını ve hiçbir kirlilik bırakmadığını kanıtladım.

Günün Çıktısı:
Atomik transaction ve rollback mimarisi kuruldu, çok adımlı operasyonlarda veri tutarlılığı birim testlerle güvenceye alındı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Eş Zamanlı Güncelleme Çakışmaları (RowVersion) ve Soft Delete Mimarisi** Tarih: **02.10.2026**

Bugün iki kullanıcının aynı gemi kaydını aynı anda güncellemeye çalışması durumunda birbirlerinin verisini ezmesi anlamına gelen 'Lost Update' eş zamanlılık problemini çözmek için çalıştım.

Ship modeline Optimistic Concurrency denetimi sağlayan '[Timestamp] public byte[] RowVersion { get; set; }' alanını ekledim. SQL Server'da rowversion tipine karşılık gelen bu alan sayesinde, bir kullanıcı eski versiyona sahip bir veriyle güncelleme göndermek istediğinde Entity Framework Core otomatik olarak DbUpdateConcurrencyException fırlatmakta ve API istemciye HTTP 409 Conflict durum kodu dönmektedir.

Günün ikinci yarısında Soft Delete (mantıksal silme) mimarisini kurdum. BaseEntity sınıfı oluşturarak tüm entity'lere IsDeleted, CreatedAt ve UpdatedAt alanlarını ekledim. ApplicationDbContext içerisinde OnModelCreating metodunda Global Query Filter tanımlayarak (HasQueryFilter(e => !e.IsDeleted)) silinmiş kayıtların varsayılan olarak tüm sorgulardan otomatik gizlenmesini sağladım.

2. aşama kapsamında yaptığım tüm performans, transaction ve concurrency çalışmalarını detaylandırarak 'docs/EF_Core_Performans_ve_Veri_Tutarliligi_Raporu.md' raporunu tamamladım.

Günün Çıktısı:
Optimistic Concurrency (RowVersion) ve Soft Delete altyapısı kuruldu, 2. Aşama performans raporu teslim edildi.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **3. Aşama: Web Güvenliği, OWASP Top 10 Analizi ve Yetki Denetimi (RBAC & IDOR)** Tarih: **05.10.2026**

Bugün staj çalışmasının 3. ve en kritik aşaması olan Siber Güvenlik ve Güvenli Yazılım Geliştirme sürecine başladık. Staj amirimle OWASP Top 10 web güvenlik standartlarını inceleyerek 'docs/OWASP_Guvenlik_Plani.md' yol haritasını çıkardık.

İlk olarak A01: Broken Access Control (Kırık Erişim Denetimi) açığını inceledim. Normalde anonim erişilmemesi gereken liman bütçesi ve MARSEC güvenlik seviyesi gibi kritik verilerin uç noktasını [Authorize(Roles = 'Admin,PortManager')] rol kısıtlaması ile yetkilendirdim. Yetkisiz isteklerin HTTP 403 Forbidden ile kesilmesini sağladım.

İkinci olarak IDOR (Insecure Direct Object Reference) zafiyetini modelledim. Bir liman müdürünün URL'deki 'portId=2' parametresini 'portId=1' yaparak başka bir limanın ticari seferlerine erişebildiğini gördüm. Bu açığı kapatmak için kullanıcının JWT token'ındaki PortId claim'i ile talep ettiği kaynak arasında eşleşme kontrolü yaptım. Yetkisiz erişim girişimlerini engelleyerek Audit Log tablosuna 'CRITICAL' önem seviyesinde kaydettim.

Günün Çıktısı:
OWASP güvenlik planı hazırlandı, Broken Access Control ve IDOR açıkları rol denetimleri (RBAC) ile tamamen kapatıldı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **SQL Injection ve XSS Açıklarının İncelenmesi ve Çözülmesi** Tarih: **06.10.2026**

Bugün arama kutuları ve kullanıcı girdileri üzerindeki en yaygın enjeksiyon açıklarını ele aldım: A03: SQL Injection ve Cross-Site Scripting (XSS).

Öncelikle dinamik string birleştirme ile yazılan güvensiz bir gemi arama sorgusunu test ettim. Arama kutusuna "' OR '1'='1" gönderildiğinde filtrenin baypas edilerek veritabanındaki tüm gemi kayıtlarının yetkisizce listelendiğini gördüm. Bu açığı gidermek için dinamik SQL cümlelerini tamamen kaldırarak Entity Framework Core parametreli LINQ sorgusunu (_context.Ships.Where(s => s.Name.Contains(term))) kullandım. SQL Server motorunun sorguyu WHERE [s].[Name] LIKE @p0 parametresi ile yürüttüğünü ve SQL Injection'ın kesin olarak engellendiğini doğruladım.

Ardından XSS açığını modelledim. Gemi açıklama alanına girilen zararlı '<script>alert('XSS')</script>' kodlarının arayüzde çalışmasını engellemek için sunucu tarafında WebUtility.HtmlEncode metodunu ve regex temizleme (sanitization) kontrollerini entegre ettim. HTML etiketlerinin pasif düz metne dönüştürülerek tarayıcıda çalışmasını engelledim.

Günün Çıktısı:
SQL Injection açığı parametreli LINQ ile, XSS açığı ise HTML encode sanitization ile tamamen giderildi.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Hassas Bilgi Sızıntısı ve Mass Assignment (Fazla Veri Atama) Önleme** Tarih: **07.10.2026**

Bugün kişisel verilerin korunması ve güvenli model bağlama konularına odaklandım: A02: Cryptographic Failures ve A08: Mass Assignment zafiyetlerini çözdüm.

Mürettebat listesi çekilirken personelin TC kimlik numarası, açık telefon numarası ve e-posta adreslerinin doğrudan entity serileştirmesi sebebiyle dışarı sızdığını (Sensitive Data Exposure) tespit ettik. KVKK/GDPR uyumu sağlamak için katmanlı DTO mimarisi kurdum: Anonim kullanıcılara sadece ad-soyad veren CrewPublicDto, liman operatörlerine telefon ve mailleri maskeli sunan CrewMaskedDto (+90 532 *** ** 33) ve sadece sistem yöneticisine açık CrewConfidentialDto geliştirdim.

Daha sonra Mass Assignment (Aşırı Atama) açığını inceledim. Kullanıcı kaydolurken JSON gövdesine 'Role: Admin' gönderdiğinde model binder'ın bunu doğrudan User entity'sine bağlayarak kullanıcıyı yönetici yaptığı açığı modelledim. Bunu önlemek için Role alanını içermeyen katı CreateUserRequest DTO'su hazırladım ve sunucu tarafında yeni kullanıcıların rolünü zorunlu olarak 'Viewer' şeklinde sabitledim.

Günün Çıktısı:
Hassas veriler için katmanlı DTO maskeleme yapısı kuruldu, Mass Assignment açığı katı request modelleriyle engellendi.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Güvenli Kimlik Doğrulama (JWT & Refresh Token) ve Hız Sınırı (Rate Limit)** Tarih: **08.10.2026**

Bugün uygulamanın kimlik doğrulama ve oturum yönetimi mimarisini kurumsal güvenlik standartlarına yükselttim: A07: Identification and Authentication Failures ve A04: Insecure Design (DoS) zafiyetlerini ele aldım.

Eski sistemde kullanılan ve süresi dolmayan statik token açığını kapattım. 15 dakikalık kısa ömürlü JWT Access Token ve 64-byte kriptografik rastgele üretilen 7 günlük Refresh Token mimarisine geçtim. Kullanıcı oturumunu korumak için tek kullanımlık token rotasyonu (Token Rotation) kodladım; her token yenilemede eski refresh token iptal edilip yenisi üretildi. Çıkış yapıldığında token'ın veritabanından silinmesini (Revocation) sağladım. Şifre hashleme için PBKDF2 algoritmasını 600.000 iterasyonla yapılandırdım.

Ardından sistemi aşırı isteklerle kilitlemeye çalışan DoS saldırılarını önlemek için ASP.NET Core RateLimiter middleware'ini entegre ettim. IP başına dakikada maksimum 120 istek, hassas oturum uç noktalarında ise 10 saniyede maksimum 5 istek kotası koydum. Kotayı aşan isteklere HTTP 429 Too Many Requests yanıtı verilmesini sağladım.

Günün Çıktısı:
15 dakikalık JWT ve Refresh Token rotasyonu kodlandı, PBKDF2 600k iterasyona çıkarıldı ve IP bazlı Rate Limiting aktif edildi.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Güvenli Dosya Yükleme, Audit Logging ve Canlı Güvenlik Laboratuvarı** Tarih: **09.10.2026**

Bugün A04: Unrestricted File Upload açığını ve güvenlik denetim günlüğü altyapısını geliştirdim. Gemi evrak yükleme ekranında sunucuya zararlı çalıştırılabilir dosyaların (.exe, .php, .sh) yüklenerek Web Shell çalıştırılmasını engellemek için katı kontroller yazdım: Yalnızca .pdf, .png ve .jpg uzantılarına izin veren uzantı beyaz listesi (whitelist) ve 5 MB boyut sınırı koydum. Yüklenen dosyalara GUID ile rastgele isimler vererek Path Traversal açıklarını önledim.

Veritabanında merkezi bir AuditLog tablosu ve servisi kurdum. Tüm yetkisiz erişim denemelerini, engellenen IDOR ve SQL Injection girişimlerini, başarısız girişleri ve hız sınırı aşımlarını IP adresi, kullanıcı adı ve zaman damgasıyla günlüğe kaydettim.

Günün ikinci yarısında frontend tarafında 10 farklı OWASP senaryosunun açık vs güvenli hallerini canlı olarak test edebildiğimiz 'Güvenlik Lab (OWASP)' arayüzünü (SecurityLabPage.tsx) geliştirdim. Kırmızı butonla açığı tetikleyip yeşil butonla savunma mekanizmasının nasıl çalıştığını sergileyen ekranı tamamladım.

Günün Çıktısı:
Güvenli dosya yükleme servisi kodlandı, merkezi Audit Logging kuruldu ve interaktif Güvenlik Laboratuvarı ekranı tamamlandı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Akıllı Sefer Planlama Sihirbazı ve Safe Manning (Güvenli Donatım) Kuralları** Tarih: **12.10.2026**

Yeni haftanın ilk gününde sisteme sektörel açıdan büyük değer katan Akıllı Sefer Planlama Sihirbazı (Smart Voyage Dispatcher) modülünü geliştirmeye başladım. Staj amirim, denizcilik regülasyonlarında gemilerin personelsiz veya eksik rütbeyle denize açılmasının kesinlikle yasak olduğunu (Safe Manning) belirtti.

Bu doğrultuda uluslararası denizcilik kurallarını kodlayan bir kural motoru geliştirdim: Konteyner ve Tanker gemilerinin sefere çıkabilmesi için gemide mutlaka aktif bir Kaptan (Master), bir Baş Mühendis (Chief Engineer) ve en az bir Güverte Zabiti (Officer) atanmış olması şartını koştum. Bu rütbelerden biri eksik olduğunda sefer başlatma (Dispatch) butonunun kilitlenmesini ve uyarı verilmesini sağladım.

Ayrıca kullanıcının işini kolaylaştırmak için, seçilen tarih aralığında ilgili limanda bağlı olan boş gemileri getiren GetAvailableShips ve dinlenme süresi uygun olan personeli listeleyen GetAvailableCrew API servis metodlarını yazdım.

Günün Çıktısı:
IMO Safe Manning asgari personel donatım kural motoru ve müsait gemi/mürettebat filtreleme servisleri kodlandı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Canlı Deniz Haritası (Live Map) ve Gerçek Rota Koordinatlarının Eklenmesi** Tarih: **13.10.2026**

Bugün projenin en görsel ve etkileyici modüllerinden biri olan Canlı Deniz Haritası (LiveMapPage.tsx) entegrasyonu üzerine çalıştım. Açık kaynaklı Leaflet.js harita kütüphanesini ve OpenStreetMap katmanlarını React projesine dahil ettim.

Türkiye'nin ana limanlarının (Ambarlı, Mersin, İzmir, Haydarpaşa, İskenderun, Samsun, Trabzon vb.) gerçek enlem ve boylam koordinatlarını sisteme girdim. Gemilerin karadan düz çizgiyle geçmesini önlemek amacıyla Türk Boğazları (İstanbul ve Çanakkale Boğazı) geçiş koridorlarını, Ege Denizi ve Akdeniz uluslararası deniz ticaret koridorlarını içeren waypoint koordinat dizilerini kodladım.

Harita üzerinde gemileri tiplerine göre (Konteyner, Tanker, Dökme Yük, Ro-Ro) farklı renk ve ikonlarla görselleştirdim. Gemilerin limanda bağlı, seyir halinde veya planlanmış olma durumlarına göre haritada canlı konumlarını ve hızlarını gösteren animasyonlu takip mekanizmasını tamamladım.

Günün Çıktısı:
Leaflet.js canlı gemi takip haritası entegre edildi, Türk Boğazları ve Akdeniz gerçek deniz seyir rotaları haritaya işlendi.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Frontend Performans İyileştirmeleri, Kod Bölme ve Hook Optimizasyonları** Tarih: **14.10.2026**

Bugün canlı harita sayfasının performansını ve sayfa açılış hızını optimize etmek için kapsamlı bir kod refaktörü gerçekleştirdim. Harita sayfasındaki rota ve koordinat dizileri binlerce satır tuttuğu için LiveMapPage.tsx bileşeni gereksiz derecede büyümüştü.

İlk olarak koordinat ve stil verilerini bağımsız modüllere ayırdım: maritimeRoutes.ts, portCoords.ts ve shipTypeStyles.ts veri dosyalarını açarak harita bileşenini hafiflettim. İkinci olarak, haritadaki filtreleme ve gemi konum hesaplamalarının her simülasyon adımında gereksiz yere baştan hesaplanmasını önlemek için React'ın useMemo ve useCallback kancalarını (hooks) uyguladım.

Son olarak, harita sayfası ağır Leaflet kütüphanesini barındırdığı için React.lazy() ve Suspense kullanarak kod bölme (Code Splitting) yaptım. Böylece harita sadece kullanıcı o menüye tıkladığında yüklendi ve ana uygulamanın ilk açılış süresi hissedilir biçimde hızlandı.

Günün Çıktısı:
Harita verileri modülerleştirildi, useMemo/useCallback ile render maliyeti düşürüldü ve React.lazy ile kod bölme sağlandı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Kargo Yönetimi Mantık Düzenlemeleri ve Liman Kontrolleri** Tarih: **15.10.2026**

Bugün kargo/yük yönetimi sayfasında operasyonel geri bildirimler doğrultusunda mantık revizyonları yaptım. Denizcilikte seyir halinde (açık denizde) olan bir gemiye fiziki olarak yeni yük yüklenemeyeceği için sistemdeki gemi seçimi mantığını güncelledim.

Yük ekleme formunda gemi seçimi yapılırken yalnızca o anda limanda bağlı ('Docked') statüsünde olan gemilerin listelenmesini sağladım. Seferde veya planlanmış durumda olan gemileri yükleme listesinden çıkararak sahadaki fiziki gerçekliğe tam uyum sağladım.

Ayrıca CargoPage.tsx sayfasına arama çubuğu ekleyerek konteyner numarası ve yük açıklamasına göre anlık filtreleme imkanı getirdim. Tabloya konşimento numarası (Bill of Lading), konteyner mühür numarası (Seal Number) ve alıcı firma (Consignee) alanlarını ekleyerek uluslararası kargo manifestosu standartlarını tamamladım.

Günün Çıktısı:
Kargo modülünde limanda bağlı gemi kontrolü sağlandı, arama çubuğu ve mühür takip alanları arayüze eklendi.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Raporlama Modülü, İstatistik Göstergeleri ve CSV Dışa Aktarımı** Tarih: **16.10.2026**

Bugün yönetimsel raporlama ve analitik göstergeler üzerinde çalıştım. Liman yöneticilerinin operasyonel durumu tek bakışta görebilmesi amacıyla backend tarafında ReportsController sınıfını kodladım.

Bu controller içerisine liman bazlı anlık doluluk oranlarını, aylık taşınan yük tonajlarını, gemi tiplerine göre sefer dağılımlarını ve aktif filo istatistiklerini hesaplayan optimize edilmiş SQL toplama (aggregation) sorguları yazdım. LINQ GroupBy ve Count/Sum operasyonlarını kullandım.

Frontend tarafında ReportsPage.tsx ekranını oluşturdum; özet KPI metrik kartları ve doluluk grafik panelleri tasarladım. Ayrıca yöneticilerin rapor verilerini Excel ve harici analiz araçlarında kullanabilmesi için tablolardaki verileri CSV formatına dönüştürüp bilgisayara indiren yardımcı dışa aktarım (Export to CSV) fonksiyonunu kodladım.

Günün Çıktısı:
Yönetim raporlama uç noktaları yazıldı, KPI gösterge kartları ve CSV dışa aktarma işlevi tamamlandı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Kurumsal Güvenlik Sıkılaştırması: API Okuma Uç Noktalarının Kilitlenmesi** Tarih: **19.10.2026**

Bugün staj sorumlumla birlikte sistemin tüm API uç noktalarını kapsayan detaylı bir güvenlik denetimi (Security Audit) gerçekleştirdik. İnceleme sırasında; yazma işlemlerinin (POST/PUT/DELETE) koruma altında olduğunu ancak bazı veri listeleme (GET) uç noktalarının anonim erişime açık kalabildiğini tespit ettik.

Kurumsal veri gizliliğini sağlamak adına CrudControllerBase taban sınıfına ve GetAll, GetPaged, GetById metodlarına [Authorize] özniteliğini ekledim. Böylece Ships, Ports, ShipVisits, Cargoes, CrewMembers ve ShipCrewAssignments verilerine geçerli JWT token olmadan erişilmesi kesin olarak engellendi.

Ayrıca denetim günlüğü uç noktalarını barındıran AuditLogsController'ı yalnızca [Authorize(Roles = 'Admin,PortManager')] rollerine tahsis ettim; önceki anonim test metodunu kaldırdım. Veri tohumlama ve sıfırlama işlemlerini yöneten DataSeedController'ı ise sadece Admin rolüne bağladım ve canlı ortamlarda kazara veri kaybı yaşanmaması için !_env.IsDevelopment() kontrolü ile HTTP 403 Forbidden güvenlik bariyeri koydum.

Günün Çıktısı:
Tüm veri okuma uç noktaları yetkilendirme altına alındı, AuditLog ve DataSeed controller'ları rol bazlı olarak sınırlandırıldı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **JWT Anahtarının Güvenli Yapılandırılması ve Ortam İzolasyonu** Tarih: **20.10.2026**

Bugün konfigürasyon ve gizli anahtar yönetimi (Secret Management) güvenliğini sıkılaştırdım. appsettings.json dosyası içerisinde açık metin olarak duran statik JWT imzalama anahtarını tamamen kaldırdım. Anahtarın üretim ortamlarında ortam değişkenlerinden (Environment Variable: Jwt__Key) okunmasını zorunlu kıldım.

Program.cs ve AuthService içerisindeki zayıf hardcoded fallback anahtarları sildim; anahtar eksik olduğunda sistemin sessizce güvensiz açılması yerine açıklayıcı bir InvalidOperationException fırlatarak durmasını (Fail-Safe Defaults) sağladım. Geliştirme ortamı için anahtarı appsettings.Development.json dosyasına izole ettim.

DbInitializer sınıfını güncelleyerek varsayılan demo kullanıcılarının (admin, operator, portmanager, viewer) yalnızca Development ortamında oluşturulmasını sağladım. Canlı ortamda ise güvenli ilk admin kurulumu parametreye bağlandı. Son olarak SecurityDemoController içerisindeki 10 adet kasıtlı açık uç noktasını Production ortamında 403 Forbidden (Mode: DISABLED) dönecek şekilde kilitledim.

Günün Çıktısı:
JWT anahtarı ortam değişkenlerine taşındı, fail-safe hata yönetimi eklendi ve demo açıkları canlı ortamda devre dışı bırakıldı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Yetkilendirme ve Güvenlik Birim Testlerinin Yazılması (144 Test)** Tarih: **21.10.2026**

Bugün uyguladığımız güvenlik sıkılaştırmalarının ve yetki kontrollerinin gerçekten çalıştığını doğrulamak amacıyla xUnit test paketini genişlettim. backend/LimanGemiTakip.Tests projesi altına yeni test sınıfları ekledim.

İlk olarak RoleAuthorizationTests.cs sınıfını yazdım. Reflection tekniklerini kullanarak tüm CRUD okuma uç noktalarının, AuditLog, DataSeed ve Reports controller'larının gerçekten [Authorize] özniteliği ve doğru rol kısıtlamalarına sahip olduğunu denetleyen 20 yeni birim testi kodladım.

Ardından SecurityScenariosTests sınıfına savunmasız demo uç noktalarının canlı ortamda 403 Forbidden döndürdüğünü doğrulayan testi, AuthServiceTests sınıfına ise eksik JWT anahtarında uygulamanın güvenle exception fırlattığını teyit eden testi ekledim.

Terminalden 'dotnet test' komutunu çalıştırarak projedeki toplam test sayısının 144'e çıktığını ve 144 testin tamamının (%100 başarı) yeşil yandığını gururla doğruladım.

Günün Çıktısı:
Birim test sayısı 144'e yükseltildi, yetkilendirme ve güvenlik senaryolarının %100 başarıyla geçtiği kanıtlandı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Kullanıcı Deneyimi İyileştirmeleri, Form Hataları ve Yükleme Efektleri** Tarih: **22.10.2026**

Bugün frontend tarafında kullanıcı deneyimini (UX) iyileştiren detaylara odaklandım. Kullanıcılar formlarda geçersiz veya eksik bilgi girdiğinde gösterilen hata bildirimlerini daha anlaşılır ve kullanıcı dostu hale getirdim.

Ağ üzerinden veri çekilirken sayfalarda boş beyaz ekran görünmesini engellemek amacıyla tablo ve istatistik kartlarının yerine iskelet yükleyiciler (Skeleton Loaders) uyguladım. Böylece veri yüklenirken kullanıcıya içeriğin geleceği hissi verildi.

Dizüstü, masaüstü ve tablet ekran çözünürlükleri için TailwindCSS responsive (duyarlı) sınıflarını (sm:, md:, lg:) gözden geçirerek ekran düzenlerinin farklı boyutlarda bozulmamasını sağladım.

Frontend klasöründe 'npm run build' ('tsc -b && vite build') komutunu çalıştırarak hiçbir TypeScript tip hatası veya derleme uyarısı olmadığını teyit ettim.

Günün Çıktısı:
Form uyarı mesajları standartlaştırıldı, iskelet yükleme efektleri eklendi ve TypeScript derlemesi sıfır hata ile doğrulandı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Tarayıcı Testleri, Kod Temizliği ve Çalışma Alanı Senkronizasyonu** Tarih: **23.10.2026**

Bugün uygulamayı farklı web tarayıcılarında (Google Chrome, Microsoft Edge, Mozilla Firefox) test ederek çapraz tarayıcı (Cross-Browser) uyumluluğunu denetledim. Leaflet canlı haritasının, modal açılır pencerelerin ve CSS geçiş efektlerinin tüm tarayıcılarda kusursuz çalıştığını gördüm.

C# ve TypeScript kodlarında detaylı bir temizlik (Code Cleanup) yaptım. Kullanılmayan kütüphane çağrılarını (using/import), gereksiz değişkenleri ve eski yorum satırlarını temizleyerek linter uyarılarını sıfırladım.

Masaüstündeki çalışma alanı ile OneDrive klasörü arasındaki dosya eşitlemelerini kontrol ettim; PowerShell senkronizasyon betiğini çalıştırarak tüm kodların, testlerin ve dokümantasyonların iki çalışma alanında da eksiksiz güncel olduğunu teyit ettim.

Günün Çıktısı:
Çapraz tarayıcı uyumluluğu sağlandı, kodlar temizlendi ve tüm çalışma alanları senkronize edildi.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Proje Dokümantasyonu, Güvenlik Raporu ve Demo Hazırlığı** Tarih: **26.10.2026**

Stajın son günlerine yaklaşırken bugün sistem dokümantasyonunu ve sunum hazırlıklarını tamamladım.

Projenin ana README.md dosyasını güncelledim; sistemin mimari şemasını, veritabanı kurulumunu, varsayılan kullanıcı rollerini ve test çalıştırma adımlarını detaylandırdım. Staj boyunca hayata geçirdiğimiz 10 OWASP güvenlik senaryosunu, kurumsal sıkılaştırmaları ve 144 birim testini içeren 'docs/OWASP_Guvenli_Yazilim_Gelistirme_Raporu.md' raporunu güncelledim.

Uygulamayı ve test ortamını tek tıkla başlatan 'baslat.bat' betiğini test ettim; hem backend API'sinin hem de frontend Vite sunucusunun saniyeler içinde sorunsuz ayağa kalktığını teyit ederek yarınki nihai staj sunumu için tüm teknik hazırlıklarımı tamamladım.

Günün Çıktısı:
Proje dokümantasyonu, güvenlik raporu ve tek tıkla başlatma betikleri nihai teslime hazır hale getirildi.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**

Bölüm: Yazılım Geliştirme Departmanı

Yapılan İş: **Staj Sonu Proje Savunması, Canlı Demo ve Stajın Tamamlanması** Tarih: **27.10.2026**

Bugün 30 iş günlük zorunlu stajımın son günüydü. Sabah saatlerinde kurumdaki staj sorumluma ve yazılım mühendisliği ekibine 'Liman Gemi Takip Sistemi' projesinin baştan sona canlı demosuyla nihai sunumunu gerçekleştirdim.

Sunumda gemi, liman, sefer ve yük CRUD yönetim ekranlarını sergiledim. Akıllı Sefer Planlama Sihirbazını çalıştırarak IMO Safe Manning personel kural motorunun eksik rütbelerde seferi nasıl engellediğini gösterdim. Canlı deniz haritası üzerinde Türk Boğazları ve Akdeniz rotalarındaki gemilerin anlık hareketini gösterdim. Güvenlik Laboratuvarında açık vs güvenli senaryoları canlı tetikledim ve terminalden 144 birim testinin tamamının geçtiğini kanıtladım.

Staj amirim ve ekip projenin mimarisini, sektörel iş kurallarını ve güvenlik olgunluğunu çok başarılı bulduklarını ifade ettiler. Staj değerlendirme görüşmemizi tamamlayarak staj defterimi ve staj belgelerimi teslim ettim ve stajımı başarıyla noktaladım.

Günün Çıktısı:
Proje nihai canlı demosu ekibe sunuldu, staj defteri ve değerlendirme belgeleri teslim edilerek staj başarıyla tamamlandı.

Stajyer İmzası: **İşyeri Bölüm Sorumlusunun İmzası:**